

Universidad Nacional
Sede Regional Brunca – Campus Coto
Bachillerato en Ingeniería de Sistemas de Información

EIF420O – Inteligencia Artificial

SentinelMail AI

Sistema inteligente de detección de phishing en correos electrónicos
y URLs mediante red neuronal, Naive Bayes, árbol de decisión y
sistema experto

Proyecto final

Estudiantes: Marco Ugalde Cortes
Jhashua Canton Ruiz
Profesor: Ing. Jeremy Elizondo Castro
Curso: Inteligencia Artificial
Ciclo lectivo: I Ciclo 2026
Fecha: 12 de junio de 2026

Costa Rica

Índice

1. Introducción	3
2. Descripción del problema	3
3. Objetivos	4
3.1. Objetivo general	4
3.2. Objetivos específicos	5
4. Justificación de las técnicas seleccionadas	5
5. Marco teórico	6
5.1. Phishing e ingeniería social	6
5.2. Clasificación supervisada	6
5.3. Red neuronal artificial	6
5.4. Naive Bayes	7
5.5. Árbol de decisión	7
5.6. Sistema experto	7
6. Diseño del sistema	8
6.1. Arquitectura general	8
6.2. Flujo de análisis	9
6.3. Módulos principales	9
6.4. Entidades del dominio	10
7. Implementación	10
7.1. Preparación de datos	10
7.2. Extracción de características	10
7.3. Red neuronal	11
7.4. Naive Bayes	12
7.5. Árbol de decisión	12
7.6. Sistema experto	13

7.7. Análisis de archivos EML y adjuntos	13
7.8. Política de clasificación final	14
8. Interfaz de usuario	15
9. Resultados y pruebas	15
9.1. Pruebas automatizadas	15
9.2. Métricas del entrenamiento	16
9.3. Caso de prueba: phishing	16
9.4. Caso de prueba: correo legítimo	17
10.Cumplimiento del enunciado	18
11.Limitaciones	19
12.Mejoras futuras	19
13.Conclusiones	20
14.Manual de usuario	20
14.1. Requisitos	20
14.2. Instalación	21
14.3. Ejecución	21
14.4. Uso básico	21
14.5. Carga de archivos EML	22
14.6. Reportes e historial	22
14.7. Pruebas	22
15.Guía para la defensa oral	22
16.Referencias bibliográficas	23

1. Introducción

El phishing es una de las amenazas más frecuentes en el uso cotidiano del correo electrónico. Consiste en engañar a una persona para que entregue credenciales, datos bancarios, información personal o acceso a sistemas por medio de mensajes que aparentan provenir de instituciones legítimas. Aunque muchas campañas de phishing son técnicamente simples, siguen siendo efectivas porque combinan elementos de ingeniería social, apariencia institucional, enlaces engañosos, urgencia psicológica y, en algunos casos, archivos adjuntos peligrosos.

El presente proyecto, denominado **SentinelMail AI**, propone una aplicación funcional para analizar correos electrónicos y URLs con el objetivo de estimar su riesgo de phishing. El sistema permite ingresar manualmente información del correo, analizar URLs, revisar cabeceras relevantes, cargar archivos con formato `.eml`, identificar señales en adjuntos y generar un resultado comprensible para el usuario. La salida principal del sistema es una clasificación de riesgo: **BAJO**, **MEDIO** o **ALTO**, acompañada por un porcentaje de riesgo y una explicación de las señales detectadas.

La motivación principal del proyecto es demostrar que las técnicas de inteligencia artificial vistas en el curso pueden aplicarse a un problema práctico y actual. La detección de phishing es adecuada para este propósito porque combina datos textuales, datos numéricos, reglas de conocimiento experto y patrones que pueden aprenderse mediante clasificación supervisada. Por esta razón, el proyecto no utiliza una única técnica, sino cuatro enfoques complementarios: una red neuronal artificial, un clasificador Naive Bayes, un árbol de decisión y un sistema experto basado en reglas de ciberseguridad.

El sistema fue desarrollado en Python y se implementó con bibliotecas estándar del lenguaje. Los algoritmos centrales de inteligencia artificial fueron programados manualmente por el grupo, sin depender de bibliotecas de aprendizaje automático como Scikit-learn, TensorFlow o PyTorch. Esta decisión responde directamente al requisito técnico del proyecto: comprender los fundamentos teóricos de las técnicas y llevarlos a una implementación propia.

2. Descripción del problema

El problema abordado consiste en clasificar correos electrónicos o enlaces como legítimos, sospechosos o probablemente maliciosos. Esta clasificación no es trivial porque un correo de phishing puede parecer normal a simple vista. Un atacante puede copiar el estilo de una institución, usar dominios visualmente parecidos, ocultar enlaces detrás de botones HTML, incluir archivos comprimidos o aprovechar cabeceras de correo inconsistentes.

Desde el punto de vista computacional, el problema puede formularse como una tarea de clasificación binaria y de puntuación de riesgo. La clase positiva corresponde a phishing y la clase negativa corresponde a correo legítimo. Sin embargo, para que la aplicación sea útil en una demostración, el resultado no se presenta únicamente como “phishing” o “legítimo”, sino como un nivel de riesgo graduado. Esto permite explicar casos intermedios en los que existen algunas señales sospechosas, pero no suficientes para afirmar con alta confianza que el mensaje es malicioso.

El análisis se basa en distintas fuentes de información:

- El asunto del correo.
- El cuerpo del mensaje.
- La URL principal.
- El remitente visible.
- Las cabeceras `Reply-To`, `Return-Path` y `Authentication-Results`.
- Los enlaces extraídos de contenido HTML o texto plano.
- Los archivos adjuntos incluidos en mensajes `.eml`.

La importancia de usar varias fuentes se debe a que el phishing no siempre se manifiesta en una sola señal. Un correo puede usar HTTPS y aun así ser malicioso; otro puede no contener adjuntos, pero solicitar credenciales; otro puede tener un texto aparentemente normal, pero fallar autenticaciones SPF, DKIM o DMARC. Por ello, SentinelMail AI combina modelos aprendidos y reglas expertas para obtener una evaluación más completa.

3. Objetivos

3.1. Objetivo general

Desarrollar una aplicación funcional capaz de analizar correos electrónicos y URLs para clasificar el riesgo de phishing mediante técnicas de inteligencia artificial implementadas manualmente.

3.2. Objetivos específicos

- Extraer características relevantes de URLs, mensajes, cabeceras y adjuntos.
- Implementar una red neuronal artificial para clasificación binaria.
- Implementar un clasificador Naive Bayes para análisis textual.
- Implementar un árbol de decisión basado en impureza Gini.
- Implementar un sistema experto con reglas de ciberseguridad.
- Combinar los resultados de las técnicas mediante una política de puntuación.
- Permitir el análisis manual de correos y la carga de archivos `.eml`.
- Generar reportes PDF y mantener historial local de análisis.
- Proporcionar una interfaz web mínima que permita demostrar el funcionamiento.
- Evaluar el sistema mediante pruebas y métricas de clasificación.

4. Justificación de las técnicas seleccionadas

Se seleccionaron cuatro técnicas porque cada una observa el problema desde una perspectiva distinta. La red neuronal permite aprender relaciones no lineales entre características numéricas. Naive Bayes es adecuado para texto porque estima probabilidades a partir de palabras frecuentes en cada clase. El árbol de decisión ofrece una clasificación por umbrales, útil para interpretar patrones. El sistema experto incorpora conocimiento humano de ciberseguridad que no depende únicamente de entrenamiento.

Esta combinación mejora la explicación del resultado final. Si todos los modelos coinciden, la decisión tiene mayor respaldo. Si un modelo detecta riesgo pero otro no, el sistema aún puede mostrar las razones específicas, como fallos de autenticación o presencia de términos sospechosos. Además, la combinación permite cumplir y superar el requisito del enunciado, que solicita al menos tres técnicas de inteligencia artificial.

Otra razón para elegir estas técnicas es que son apropiadas para una implementación manual. Cada algoritmo puede programarse desde sus fundamentos: inicialización de pesos y retropropagación en la red neuronal, conteo de palabras y suavizado de Laplace en Naive Bayes, cálculo de impureza Gini en el árbol, y reglas ponderadas en el sistema experto. Esto permite demostrar dominio del código durante la defensa oral.

5. Marco teórico

5.1. Phishing e ingeniería social

El phishing es una técnica de fraude digital que busca inducir a una víctima a realizar una acción insegura. Esta acción puede consistir en abrir un enlace, descargar un archivo, introducir credenciales, confirmar datos personales o realizar un pago. En muchos casos, el atacante utiliza mensajes que imitan bancos, universidades, plataformas de correo, empresas de paquetería o servicios de entretenimiento.

Las señales comunes de phishing incluyen lenguaje de urgencia, amenazas de suspensión, solicitudes de contraseña, enlaces a dominios extraños, uso de direcciones IP en lugar de dominios, archivos adjuntos con extensiones ejecutables y errores en mecanismos de autenticación de correo. Aunque ninguna señal individual garantiza que un correo sea malicioso, la acumulación de señales aumenta el riesgo.

5.2. Clasificación supervisada

La clasificación supervisada es una tarea de aprendizaje automático en la que se entrena un modelo con ejemplos etiquetados. Cada ejemplo contiene datos de entrada y una etiqueta esperada. En este proyecto, la etiqueta 1 representa phishing y la etiqueta 0 representa correo legítimo.

Durante el entrenamiento, los modelos aprenden patrones a partir de los ejemplos. Durante la predicción, reciben un nuevo correo o URL y devuelven una probabilidad o decisión. Para validar el rendimiento, los datos se separan en conjunto de entrenamiento y conjunto de prueba.

5.3. Red neuronal artificial

Una red neuronal artificial está formada por unidades llamadas neuronas, organizadas en capas. Cada neurona calcula una suma ponderada de sus entradas y aplica una función de activación. En este proyecto se utiliza la función sigmoide, definida como:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

La sigmoide produce valores entre 0 y 1, lo que permite interpretar la salida como probabilidad de phishing. La red implementada tiene una capa de entrada, una capa oculta

y una neurona de salida. El entrenamiento se realiza mediante retropropagación, ajustando pesos y sesgos para reducir el error entre la predicción y la etiqueta real.

5.4. Naive Bayes

Naive Bayes es un clasificador probabilístico basado en el teorema de Bayes. En clasificación textual, se usa para estimar la probabilidad de una clase dado un conjunto de palabras. Su nombre incluye la palabra “naive” porque asume independencia condicional entre términos, una simplificación que en la práctica suele funcionar bien para texto.

La idea general es calcular:

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

Donde C es la clase y X representa las palabras del mensaje. Para evitar probabilidades cero cuando aparece una palabra nueva, se utiliza suavizado de Laplace. En el proyecto, Naive Bayes trabaja con tokens extraídos del asunto, cuerpo, URL, remitente y nombres de adjuntos.

5.5. Árbol de decisión

Un árbol de decisión clasifica ejemplos mediante divisiones sucesivas de las características. Cada nodo interno evalúa una condición, por ejemplo si una característica es menor que cierto umbral. Cada hoja contiene una decisión o probabilidad.

Para construir el árbol se busca la división que mejor separe las clases. SentinelMail AI utiliza impureza Gini, que mide qué tan mezcladas están las clases en un grupo. Una impureza baja indica que el grupo contiene principalmente ejemplos de una clase.

$$Gini = 1 - p^2 - (1 - p)^2$$

Donde p es la proporción de ejemplos positivos. El árbol implementado limita su profundidad para evitar crecimiento excesivo y mantener la clasificación interpretable.

5.6. Sistema experto

Un sistema experto representa conocimiento humano mediante reglas. A diferencia de los modelos entrenados, no aprende pesos a partir de datos, sino que evalúa condiciones

diseñadas por especialistas. En este proyecto, las reglas se basan en señales comunes de phishing y seguridad de correo.

Ejemplos de reglas son:

- Si la URL no usa HTTPS, aumentar el riesgo.
- Si el dominio es una dirección IP, aumentar el riesgo.
- Si el mensaje solicita contraseña o token, aumentar el riesgo.
- Si SPF, DKIM o DMARC fallan, aumentar el riesgo.
- Si un adjunto tiene doble extensión, aumentar el riesgo.

El sistema experto es valioso porque permite explicar directamente al usuario por qué un mensaje fue considerado riesgoso.

6. Diseño del sistema

6.1. Arquitectura general

El proyecto se organizó siguiendo una separación por capas inspirada en arquitectura limpia. Esta estructura facilita mantener separada la lógica del dominio, los casos de uso, la infraestructura y la interfaz.

Capa	Responsabilidad
Dominio	Define entidades principales, resultados y política de clasificación.
Aplicación	Coordina el caso de uso principal de análisis.
Infraestructura	Implementa modelos de IA, extracción de características, datasets, parser EML, historial y reportes.
Presentación	Expone la interfaz web, endpoints HTTP, archivos estáticos y plantillas HTML.

La separación permite explicar claramente el sistema durante la defensa. La interfaz no contiene los algoritmos de IA; solo envía datos al caso de uso. El caso de uso no sabe cómo se dibuja la pantalla; únicamente coordina modelos y devuelve un resultado. Los modelos se ubican en infraestructura porque son detalles técnicos de implementación.

6.2. Flujo de análisis

El flujo funcional del sistema es el siguiente:

1. El usuario ingresa datos del correo o carga un archivo `.eml`.
2. El servidor construye un objeto `EmailAnalysisRequest`.
3. El extractor convierte el correo en características numéricas y texto tokenizado.
4. La red neuronal calcula una probabilidad de phishing.
5. Naive Bayes calcula una probabilidad textual de phishing.
6. El árbol de decisión calcula una probabilidad según umbrales aprendidos.
7. El sistema experto suma reglas activadas y produce razones explicables.
8. La política de riesgo combina los cuatro puntajes.
9. El sistema muestra decisión, porcentaje, detalles, reglas, métricas y reporte PDF.

6.3. Módulos principales

Archivo	Función
<code>codigo/app.py</code>	Punto de entrada de la aplicación web.
<code>domain/entities.py</code>	Define entidades como solicitud, adjuntos, indicadores y resultado.
<code>domain/policies.py</code>	Combina puntajes y decide nivel de riesgo.
<code>application/ analyze_email.py</code>	Caso de uso principal.
<code>ai/ feature_extractor.py</code>	Extrae características numéricas y tokens.
<code>ai/ neural_network.py</code>	Red neuronal manual.
<code>ai/ naive_bayes.py</code>	Clasificador Naive Bayes manual.
<code>ai/ decision_tree.py</code>	Árbol de decisión manual.
<code>ai/ expert_system.py</code>	Sistema experto de reglas.
<code>email_parser.py</code>	Parser de correos <code>.eml</code> y adjuntos.
<code>external_dataset.py</code>	Carga datasets externos.

<code>reports.py</code>	Genera reportes PDF.
<code>server.py</code>	Servidor HTTP e interfaz.

6.4. Entidades del dominio

El dominio define estructuras de datos independientes de la interfaz. La entidad principal de entrada es `EmailAnalysisRequest`, que contiene asunto, cuerpo, URL, remitente, cabeceras, enlaces, adjuntos y nombre de origen. La salida principal es `AnalysisResult`, que contiene decisión, nivel, puntaje final, puntajes por técnica, razones, indicadores coincidentes, resumen del correo, características y métricas.

Esta estructura permite que el mismo análisis funcione tanto desde la interfaz web como desde el endpoint JSON `/api/analyze`.

7. Implementación

7.1. Preparación de datos

El sistema entrena sus modelos al iniciar la aplicación. Primero carga un dataset simulado realista desde `SimulatedPhishingDataset`. Este dataset contiene ejemplos legítimos y ejemplos de phishing con asuntos, URLs y cuerpos de mensaje. Además, puede cargar datasets externos desde `codigo/data/datasets` en formatos CSV, JSON, JSONL o EML.

Cada fila del dataset se convierte en dos representaciones:

- Un vector numérico de características para red neuronal y árbol.
- Un texto tokenizado para Naive Bayes.

Luego se realiza una división de entrenamiento y prueba con una proporción de 75 % para entrenamiento y 25 % para prueba. La semilla fija permite que la separación sea reproducible.

7.2. Extracción de características

El extractor genera 23 características. Estas características fueron diseñadas para capturar señales comunes de phishing. Algunas son binarias, como si la URL usa HTTPS o si el dominio contiene punycode. Otras son proporciones normalizadas, como longitud de URL, cantidad de enlaces o cantidad de palabras sospechosas.

No.	Característica
1	La URL usa HTTPS.
2	Longitud de la URL normalizada.
3	Cantidad de puntos en la URL.
4	Cantidad de guiones.
5	Uso de dirección IP como dominio.
6	TLD sospechoso.
7	Presencia de arroba en la URL.
8	Cantidad de dígitos en la URL.
9	Cantidad de enlaces.
10	Palabras de urgencia.
11	Palabras relacionadas con credenciales.
12	Palabras financieras.
13	Uso de acortadores.
14	Presencia de formularios HTML.
15	Extensiones peligrosas mencionadas.
16	Proporción de palabras sospechosas.
17	Cantidad de adjuntos.
18	Diferencia entre Reply-To y From .
19	Diferencia entre Return-Path y From .
20	Fallo SPF, DKIM o DMARC.
21	Cantidad de enlaces declarados.
22	Uso de punycode.
23	Señales riesgosas en adjuntos.

7.3. Red neuronal

La red neuronal se implementó en la clase `ManualNeuralNetwork`. Su arquitectura es simple pero suficiente para clasificación binaria: una capa de entrada con 23 características, una capa oculta de 10 neuronas y una neurona de salida.

El método `_forward` calcula la propagación hacia adelante. Primero obtiene las activaciones de la capa oculta y luego calcula la salida final. El método `train` ejecuta 900 épocas de entrenamiento y ajusta los pesos mediante retropropagación.

Listing 1: Esquema de predicción de la red neuronal

```
hidden = sigmoid(w1 * x + b1)
output = sigmoid(w2 * hidden + b2)
```

El error utilizado es pérdida logarítmica binaria. El gradiente de la salida se calcula como la diferencia entre predicción y etiqueta real. Luego se actualizan los pesos de salida y los pesos de la capa oculta.

7.4. Naive Bayes

El clasificador `ManualNaiveBayes` utiliza una bolsa de palabras. Durante el entrenamiento, cuenta cuántos documentos pertenecen a cada clase y cuántas veces aparece cada palabra en cada clase. También construye el vocabulario total.

Durante la predicción, calcula un puntaje logarítmico para cada clase. Se usa suavizado de Laplace sumando 1 al conteo de cada palabra. Al final, los puntajes se transforman en una probabilidad normalizada de phishing.

Listing 2: Idea central del suavizado de Laplace

```
count = word_counts[label][token] + 1
probability = count / (total_words[label] + vocabulary_size)
```

Este modelo es especialmente útil para detectar vocabulario asociado a phishing, como “urgente”, “contraseña”, “token”, “tarjeta”, “premio”, “bloqueada” o “reembolso”.

7.5. Árbol de decisión

El árbol de decisión se implementó en `ManualDecisionTree`. El entrenamiento construye un árbol binario de manera recursiva. En cada nodo se prueban características y umbrales posibles. La mejor división es aquella que minimiza la impureza Gini ponderada de los grupos izquierdo y derecho.

La predicción recorre el árbol desde la raíz hasta una hoja. Si el valor de la característica es menor que el umbral, se avanza hacia la rama izquierda; en caso contrario, hacia la derecha. La hoja devuelve una probabilidad de phishing basada en la proporción de ejemplos positivos.

El árbol tiene una profundidad máxima de 5 y un tamaño mínimo de nodo de 2, lo cual evita un crecimiento excesivo.

7.6. Sistema experto

El sistema experto se implementó en **SecurityExpertSystem**. A diferencia de los modelos anteriores, no se entrena con datos. Evalúa reglas diseñadas con conocimiento de ciberseguridad y suma puntos de riesgo.

Entre las reglas más importantes están:

- URL sin HTTPS: suma 15 puntos.
- Dirección IP en el dominio: suma 18 puntos.
- TLD sospechoso: suma 10 puntos.
- Urgencia en el texto: suma 12 puntos.
- Solicitud de credenciales: suma 14 puntos.
- Señuelo financiero: suma 10 puntos.
- Fallo SPF o DKIM: suma 18 puntos.
- Fallo DMARC: suma 20 puntos.
- Doble extensión en adjuntos: suma 10 puntos.
- Posibles macros en adjuntos: suma 12 puntos.

El puntaje del sistema experto se limita a un máximo de 100. Además, cada regla activada produce una razón que se muestra al usuario. Esto aporta interpretabilidad.

7.7. Análisis de archivos EML y adjuntos

El parser de correos se implementó con la biblioteca estándar **email**. Cuando el usuario carga un archivo **.eml**, el sistema extrae asunto, cuerpo en texto plano, contenido HTML, enlaces, cabeceras y adjuntos.

Para cada adjunto se calcula:

- Nombre de archivo.
- Tipo de contenido.
- Tamaño.

- Hash SHA-256.
- Extensión.
- Presencia de doble extensión.
- Sospecha de macros.
- Señales de ejecutable de Windows.

El hash SHA-256 permite identificar el archivo sin ejecutarlo. El sistema no abre ni ejecuta adjuntos, lo cual es importante desde el punto de vista de seguridad.

7.8. Política de clasificación final

La política de riesgo combina los puntajes de las cuatro técnicas. La fórmula utilizada es:

$$R = 0,36N + 0,24B + 0,18A + 0,22E$$

Donde:

- N es el puntaje de la red neuronal.
- B es el puntaje de Naive Bayes.
- A es el puntaje del árbol de decisión.
- E es el puntaje del sistema experto.

La decisión se determina así:

Puntaje final	Nivel	Decisión
0 a 39.99	BAJO	Probablemente legítimo
40 a 69.99	MEDIO	Sospechoso
70 a 100	ALTO	Probable phishing

8. Interfaz de usuario

El sistema incluye una interfaz web local ejecutada mediante un servidor HTTP en Python. Para iniciar la aplicación se ejecuta:

```
python codigo/app.py
```

Luego se abre el navegador en:

`http://127.0.0.1:8000`

La interfaz permite ingresar asunto, URL, cuerpo del correo, remitente, **Reply-To**, **Return-Path** y **Authentication-Results**. También permite cargar archivos **.eml**. Al analizar un correo, la aplicación muestra el resultado final, los puntajes por técnica, las razones encontradas, indicadores coincidentes, resumen de cabeceras, adjuntos detectados, métricas del modelo y un enlace al reporte PDF generado.

Además, la aplicación cuenta con rutas de ejemplo:

- `/sample/phishing`
- `/sample/legit`
- `/sample-eml/phishing-completo`

También existe un endpoint JSON en:

`http://127.0.0.1:8000/api/analyze`

Esto permite demostrar que la lógica del sistema puede ser usada tanto por la interfaz visual como por otros clientes.

9. Resultados y pruebas

9.1. Pruebas automatizadas

El proyecto incluye pruebas de humo en `codigo/tests/test_detector.py`. Estas pruebas validan tres comportamientos importantes:

- Un ejemplo de phishing debe clasificarse como riesgo alto.

- Un ejemplo legítimo debe clasificarse como riesgo bajo.
- Un archivo `.eml` debe parsearse correctamente, incluyendo adjuntos y autenticación.

La ejecución de pruebas produjo el siguiente resultado:

```
Ran 3 tests in 1.630s
OK
```

Esto demuestra que los casos principales del detector funcionan correctamente.

9.2. Métricas del entrenamiento

Durante la verificación del sistema se obtuvieron las siguientes métricas:

Métrica	Valor
Total de ejemplos	60
Ejemplos de entrenamiento	45
Ejemplos de prueba	15
Ejemplos simulados	60
Ejemplos externos	0
Exactitud de entrenamiento	100.0 %
Exactitud de prueba	100.0 %
Verdaderos positivos	9
Verdaderos negativos	6
Falsos positivos	0
Falsos negativos	0

La exactitud alta se explica porque el dataset educativo contiene señales claras y fue diseñado para demostrar el funcionamiento del sistema. En un entorno real, se esperaría que la exactitud varíe y que sea necesario entrenar con corpus más amplios y diversos.

9.3. Caso de prueba: phishing

Se probó el siguiente caso:

- Asunto: Cuenta bloqueada urgente.

- URL: `http://seguridad-banco.example.verify-login.ru/acceso`
- Mensaje: Urgente: su cuenta será suspendida. Verifique usuario, contraseña y token.

El resultado fue:

Campo	Valor
Decisión	Probable phishing
Nivel	ALTO
Riesgo final	98.02 %
Red neuronal	100.0 %
Naive Bayes	100.0 %
Árbol de decisión	100.0 %
Sistema experto	91.0 %

Las principales razones activadas fueron: la URL no usa HTTPS, el dominio usa terminación sospechosa, el dominio contiene subdominios o guiones, el mensaje usa urgencia y solicita credenciales.

9.4. Caso de prueba: correo legítimo

Se probó el siguiente caso:

- Asunto: Aviso de mantenimiento.
- URL: `https://hosting.example/status`
- Mensaje: El servicio tendrá una ventana de mantenimiento programada el sábado.

El resultado fue:

Campo	Valor
Decisión	Probablemente legítimo
Nivel	BAJO
Riesgo final	0.0 %
Red neuronal	0.01 %
Naive Bayes	0.0 %
Árbol de decisión	0.0 %

Sistema experto

0.0 %

El sistema no activó reglas críticas de phishing, lo cual coincide con el comportamiento esperado.

10. Cumplimiento del enunciado

El enunciado solicita una aplicación funcional que resuelva un problema específico utilizando al menos tres técnicas de inteligencia artificial cubiertas en el curso. SentinelMail AI cumple este requisito porque resuelve el problema de detección de phishing y aplica cuatro técnicas diferentes.

Requisito del enunciado	Cumplimiento en el proyecto
Resolver un problema claramente definido con IA	Se resuelve la detección de phishing en correos y URLs.
Implementar al menos 3 técnicas de IA	Se implementan 4: red neuronal, Naive Bayes, árbol de decisión y sistema experto.
Funcionar con datos reales o simulados realistas	Usa dataset simulado realista y puede cargar datasets externos.
Incluir interfaz de usuario mínima	Incluye interfaz web local.
Algoritmos centrales implementados manualmente	Los modelos se implementaron sin bibliotecas de aprendizaje automático.
Documentación técnica clara	Este informe documenta problema, diseño, implementación, resultados y uso.
Resultados verificados	Se ejecutaron pruebas automatizadas y casos manuales.
Manual de usuario	Se incluye en una sección específica.

En cuanto al formato de entrega, el enunciado solicita una carpeta de código, una carpeta de documentación, una carpeta de presentación y un archivo README. El repositorio contiene la carpeta `codigo`, la carpeta `documentacion` y el archivo `README.txt`. Para la entrega final, se recomienda agregar una carpeta `presentacion` con las diapositivas de la defensa y comprimir todo en formato ZIP con el nombre solicitado por el profesor.

11. Limitaciones

Aunque el sistema cumple los objetivos académicos, tiene limitaciones importantes:

- El dataset incluido es simulado, aunque realista. Esto facilita la demostración, pero no sustituye un corpus real amplio.
- No consulta servicios externos de reputación de dominios.
- No ejecuta adjuntos en sandbox; solo realiza análisis estático.
- Puede generar falsos positivos si un correo legítimo usa lenguaje urgente.
- Puede generar falsos negativos si un phishing está escrito de forma muy cuidadosa y sin señales evidentes.
- El reporte PDF es funcional, pero básico en diseño visual.

Estas limitaciones no invalidan el proyecto; más bien delimitan su alcance académico. El objetivo no es reemplazar una plataforma profesional de seguridad, sino demostrar la aplicación de técnicas de inteligencia artificial a un problema realista.

12. Mejoras futuras

Como mejoras futuras se proponen:

- Entrenar con datasets públicos más grandes y diversos.
- Agregar validación con dominios reales y reputación en línea.
- Incorporar análisis de similitud visual para detectar dominios homógrafos.
- Analizar adjuntos en un entorno sandbox controlado.
- Mejorar el diseño del reporte PDF.
- Agregar explicabilidad por característica para la red neuronal.
- Permitir exportar resultados en CSV o JSON desde la interfaz.
- Agregar autenticación de usuarios si se desea usar el sistema en una red institucional.

13. Conclusiones

SentinelMail AI demuestra que es posible construir una aplicación funcional de detección de phishing utilizando técnicas de inteligencia artificial implementadas manualmente. El proyecto integra cuatro enfoques diferentes y los combina en una política de riesgo clara. Esta integración permite analizar texto, características numéricas, reglas de seguridad y estructura de correos electrónicos.

La red neuronal aprende patrones numéricos asociados al riesgo. Naive Bayes identifica vocabulario frecuente en mensajes de phishing. El árbol de decisión aporta clasificación por umbrales e interpretación estructurada. El sistema experto incorpora reglas explícitas de ciberseguridad y genera explicaciones comprensibles para el usuario.

El sistema cumple con los requisitos técnicos principales del enunciado: tiene un problema definido, implementa más de tres técnicas de IA, funciona con datos simulados realistas, posee interfaz web, incluye pruebas, genera reportes y mantiene historial de análisis. Además, los algoritmos centrales fueron programados sin bibliotecas externas de aprendizaje automático, lo cual evidencia comprensión de los fundamentos.

Como aprendizaje principal, el proyecto muestra que la inteligencia artificial aplicada no depende únicamente de modelos complejos. En problemas de seguridad, la combinación de modelos estadísticos, aprendizaje supervisado y reglas expertas puede ofrecer resultados más explicables y útiles que una única técnica aislada.

14. Manual de usuario

14.1. Requisitos

Para ejecutar el sistema se requiere:

- Python 3.10 o superior.
- Sistema operativo Windows, Linux o macOS.
- Navegador web moderno.
- No se requieren bibliotecas externas para ejecutar la aplicación.

14.2. Instalación

El usuario debe descomprimir el proyecto y abrir una terminal en la carpeta raíz. La estructura esperada incluye:

- `codigo/`
- `documentacion/`
- `README.txt`

14.3. Ejecución

Desde la carpeta raíz del proyecto, ejecutar:

```
python codigo/app.py
```

El sistema mostrará un mensaje similar a:

```
Detector iniciado en http://127.0.0.1:8000  
Presione Ctrl+C para detener.
```

Luego se debe abrir el navegador en:

```
http://127.0.0.1:8000
```

14.4. Uso básico

Para analizar un correo manualmente:

1. Escribir o pegar el asunto.
2. Ingresar la URL principal.
3. Pegar el cuerpo del mensaje.
4. Opcionalmente completar remitente, `Reply-To`, `Return-Path` y resultados de autenticación.
5. Presionar el botón de análisis.
6. Revisar el nivel de riesgo, puntajes por técnica y razones encontradas.

14.5. Carga de archivos EML

Para analizar un correo real exportado:

1. Seleccionar un archivo con extensión `.eml`.
2. Enviar el formulario.
3. El sistema extraerá asunto, cuerpo, enlaces, cabeceras y adjuntos.
4. Se mostrará el resultado de riesgo.

14.6. Reportes e historial

Cada análisis genera un reporte PDF en la carpeta `codigo/data/reports`. También se guarda un resumen en `codigo/data/history/analysis_history.json`. El historial conserva los últimos análisis para consulta desde la interfaz.

14.7. Pruebas

Para ejecutar las pruebas automatizadas:

```
python -m unittest discover codigo/tests
```

El resultado esperado es que las pruebas finalicen con OK.

15. Guía para la defensa oral

Para una exposición de 15 a 20 minutos, se recomienda dividir la presentación de la siguiente manera:

Tiempo	aproxima-	Contenido
do		
2 minutos		Presentación del problema de phishing y motivación.
2 minutos		Objetivo general y alcance del sistema.
4 minutos		Explicación de las cuatro técnicas de IA.
3 minutos		Arquitectura y flujo de análisis.
4 minutos		Demostración en vivo con caso phishing y caso legítimo.

2 minutos	Resultados, métricas y pruebas.
2 minutos	Conclusiones, limitaciones y mejoras futuras.

Una respuesta breve para explicar el proyecto podría ser:

SentinelMail AI es una aplicación web local que analiza correos electrónicos y URLs para estimar riesgo de phishing. El sistema combina cuatro técnicas de inteligencia artificial implementadas manualmente: red neuronal, Naive Bayes, árbol de decisión y sistema experto. Cada técnica evalúa señales distintas y luego una política de riesgo combina los puntajes para producir una decisión final explicable.

Si el profesor pregunta por qué se usan cuatro técnicas, la respuesta sugerida es:

Porque el phishing no se detecta con una sola señal. La red neuronal aprende patrones numéricos, Naive Bayes analiza lenguaje, el árbol clasifica por umbrales y el sistema experto incorpora reglas de ciberseguridad. Al combinarlas, el sistema obtiene una evaluación más completa y explicable.

Si el profesor pregunta cómo se programaron los algoritmos, la respuesta sugerida es:

Los algoritmos centrales se implementaron manualmente. La red neuronal usa sigmoide, pérdida logarítmica y retropropagación; Naive Bayes usa conteo de tokens, probabilidad por clase y suavizado de Laplace; el árbol usa divisiones por umbral e impureza Gini; el sistema experto usa reglas ponderadas de seguridad.

16. Referencias bibliográficas

Referencias

- [1] Russell, S. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. Pearson.
- [2] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.
- [3] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
- [4] Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*. MIT Press.

- [5] OWASP Foundation. (2024). *Phishing and social engineering security awareness resources*. OWASP.
- [6] National Institute of Standards and Technology. (2020). *Security and Privacy Controls for Information Systems and Organizations*. NIST Special Publication 800-53.
- [7] Kitterman, S. (2014). *Sender Policy Framework (SPF) for Authorizing Use of Domains in Email*. RFC 7208.